Empório da Música
Generative AI Support Agent

AI Engineer technical challenge • Artefact
Candidate: Silas Maia de Oliveira • Draft for candidate review

Problem and implemented scope

Mel assists a fictitious musical-instrument store with catalog discovery, product availability, pricing, payment options, order status and store policies. Responses are in Brazilian Portuguese; technical deliverables are in English. A shared Python agent supports a local chat, CLI and an official Meta WhatsApp Cloud API adapter.

Architecture

A tool-calling LLM selects typed Python tools. SQLite owns structured business facts; a local lexical retriever supplies PDF sections and physical page references. Prices, discounts, installments and ownership checks are deterministic Python operations. The model has no arbitrary SQL, recipient-selection or authentication-mutation capability.

Data and retrieval

The workbook contains 65 products, 9 categories, 50 customers, 20 orders, 22 order items and 25 promotions. All ten tabs are retained as provenance. Six relational tables preserve explicit joins and foreign keys. A SQL catalog view avoids a stale duplicate. The small policy manual is retrieved with accent normalization and domain aliases; an external vector service was unnecessary for this prototype.

Conversational experience

Mel keeps session-specific context, responds directly and consults tools for commercial facts. Follow-up questions such as “And Pix?” reuse the discussed product. Unavailable instruments lead to alternatives; returns require the missing purchase channel and actual receipt date. The agent does not claim to execute refunds, reserve stock or notify a human externally.

Commercial interpretations

Only active promotions apply; Pix does not stack with promotional discounts. Detailed installment bands take precedence over the summary table. The exact R$500 shipping boundary and genuinely ambiguous eligibility require confirmation. Historical order totals remain unchanged. Product-specification conflicts are preserved without blocking unrelated price and stock answers.

Integration, Validation and Limits

Official Meta adapter

A FastAPI webhook validates the GET handshake and raw-body HMAC-SHA256. Incoming message IDs are durably deduplicated before acknowledgement. A separate worker handles the LLM and outbound API calls, respecting per-conversation ordering. Free text outside the service window is blocked. Templates require explicit operator-confirmed approval and configuration.

Delivery semantics

The queue distinguishes pending, processing, sending, sent, simulated, blocked, failed and uncertain states. Rate limits have bounded retries. A timeout or crash during sending may have an unknown external outcome; it is not blindly re-sent. Exactly-once WhatsApp delivery is not claimed. The supported deployment baseline is one worker.

Executed evidence

46 deterministic pytest cases passed. The 25 numbered notebook blocks executed successfully for local checks; external LLM and Meta blocks were explicitly skipped. The local Streamlit interface passed a startup smoke test. Tests cover source integrity, joins, monetary rules, retrieval, authorization, session isolation, signed webhooks, deduplication, recovery and mocked HTTP delivery. Five example conversations are explicitly scripted simulations with real tool execution.

Unvalidated external dependencies

No real model call or WhatsApp message was sent during this build. Credentials, account permissions, approved templates, a supported Graph API version and a reachable HTTPS webhook must be configured by the operator. Docker files are included but a Docker build was not executed. The supplied database is a snapshot, not an ERP or carrier integration.

Security and next steps

Order tools enforce backend-owned customer identity. Meta sessions remain anonymous until a trusted verification process creates a binding; the UI selector is demonstration-only. Production work includes real identity verification, data retention/encryption, delivery reconciliation, stronger concurrency control and live adversarial/model-quality evaluation. Prompt-level behavior is not proven by deterministic mocks.

AI assistance and submission

ChatGPT/Codex assisted architecture, coding, tests and documentation. The candidate must review and understand the implementation and validate live integrations. Incremental Git commits record actual build stages, not fabricated history. Repository publication and submission were not performed. Add the final public repository URL before delivering this English PDF and the repository link through the requested channels.